Repository: https://github.com/sunitazenith07-lang/ShuttleOps-Open-Source-Evaluation

SmartDispatch: Delivery Partner Assignment & ETA Engine

Individual 24-hour challenge | Python 3.10+ | Synthetic data only

Scenario

A food-delivery service assigns orders to delivery partners. Partner availability, existing workload, restaurant preparation time and travel estimates affect the delivery ETA. Build the decision logic; do not build a mobile app, map UI or live GPS integration.

This is a simulated challenge inspired by common logistics problems, not any company's internal code.

Required behaviour

1. Validate order ID, restaurant ID, destination zone and non-negative preparation/travel times.

2. Consider only active partners below their order capacity. A candidate cannot receive a reliable ETA if any required travel-time entry is missing.

3. Calculate pickup time as current time + partner-to-restaurant travel + remaining workload. Calculate ETA as pickup time + restaurant preparation + restaurant-to-customer travel.

4. Pick the eligible candidate with the lowest ETA. Break ETA ties by partner ID in ascending lexicographic order.

5. Missing travel data must never be treated as zero. If no valid candidate remains, leave the order unassigned and explain why.

6. Process a cancellation event only once so a duplicate cancellation does not release workload/capacity twice.

7. Return deterministic results and handle empty lists, unknown IDs, malformed records and negative time values safely.

Setup

Run: `python -m pip install -r requirements-dev.txt`, then `python -m pytest -q`, then `python -m main`.

Repository reading checklist and dataset

Before editing, read `README.md` and `STUDENT_START_HERE.md`, then inspect the whole repository—not only this PDF. Read `CONTRIBUTING.md`, `.github/PULL_REQUEST_TEMPLATE.md`, the test/template documents, `service.py`, `models.py`, `fixtures.py`, `main.py`, every file under `tests/`, and the workflow files under `.github/workflows/`.

The expanded synthetic dataset is in `data/records.json`: **180 orders, 60 delivery-partner records, 387 travel-time records, and 177 cancellation events**, plus an edge-case catalog. Read `data/README.txt` for its schema. Some

travel routes are deliberately missing; the dataset also covers full capacity, inactive partners, ETA boundary values, tie-breaking, duplicate cancellations and invalid inputs. Convert the JSON travel-time rows into the tuple-key mapping expected by `service.py`. The offline demo uses `fixtures.py`; the large JSON is additional investigation and test material.

Run the baseline tests before editing and record their actual result. Do not delete, skip, weaken or bypass tests.

Work to submit

- Read the source and run the starter tests before editing.
- Treat the required behaviour above as the acceptance contract. Investigate failing assertions, locate the root cause and fix the logic instead of hard-coding fixture results.
- Add regression tests and document at least five edge cases in `TESTING.md`.
- Submit one pull request to the original repository containing your code and tests. Do not merge it.
- Include `DISPATCH_NOTES.md` with one worked ETA, one no-eligible-partner example and three edge cases.

Mandatory problem-solving documentation — all three files required

Submit these three completed Markdown files at the repository root in your pull request. They are evaluated; code alone is not a complete submission.

- `approach.md` — record your starting test result, the code/data flow you explored, key hypotheses, experiments you ran, implementation strategy, and why you chose it.
- `challenges.md` — **COMPULSORY**. Describe concrete obstacles or confusing behaviours, what you tried, evidence from tests/logs, how you resolved them, and any unresolved limitations. If you did not get stuck, state that honestly and describe the hardest edge case you deliberately tested; do not invent a struggle.
- `optimizations.md` — describe optimizations considered or implemented, why you chose or rejected them, correctness/readability/performance trade-offs, and future improvements. Separate measured improvements from ideas you did not measure.

Keep the notes concise, specific and evidence-based. Describe your actual work, tests and technical decisions; do not provide private hidden chain-of-thought or a stream-of-consciousness transcript. Generic or copied text does not demonstrate your process. Each file is worth 5 marks. A missing or empty file receives 0/5 for that component and makes the documentation requirement incomplete.

Optional stretch work

Choose at most one after the core works: batch-assign orders in supplied priority order; return an explanation for each candidate; or recalculate ETA after a preparation-time change.

Testing and GitHub Actions

Some visible tests are expected to fail on the untouched starter due to incomplete behaviour. The exact test count is not fixed. GitHub Actions runs on pushes and pull-request updates and reports visible test counts. A red check before fixes is expected. Do not remove, skip, weaken or bypass tests or edit CI to evade failures.

Individual-work policy

AI chatbots, AI coding assistants, generated patches/tests and another person's solution-writing/debugging are prohibited. Ordinary non-AI documentation is allowed. Do not paste challenge code/data into external services. You must explain your work and make a small live change if asked. Code walkthroughs can support integrity checks, but no automated detector proves AI use.

Rubric — 100 marks

- Codebase understanding and ETA trace: 10
- Assignment logic and deterministic tie-breaking: 20
- Edge-case safety, capacity consistency and cancellation handling: 20
- Regression tests and edge cases: 15
- Code quality and maintainability: 5
- Required problem-solving documentation: 15 (approach.md: 5, challenges.md: 5, optimizations.md: 5)
- Pull request, dispatch notes and live explanation: 15

The three process documents are assessed for specific, honest, evidence-based technical reflection, not writing style.